GIT-001 – Gyakorló feladatsor

Tematikus feladatsor – verziókezelés Gittel

Rész	Feladat	Pont/feladat	Összesen
I. Git alapok és mindennapi munka	25	2	50
II. Ágak, együttműködés és haladó eszközök	25	2	50
Összesen	**50**		**100**

Tudnivalók

• A feladatsor megoldására 75 perc áll rendelkezésre.

• Minden feladatnál pontosan egy válasz helyes. Ha a választ kihagyja, vagy egynél több lehetőséget jelöl meg, az adott feladatra 0 pont jár.

• A megoldáshoz íróeszközön és nem programozható számológépen kívül más segédeszköz nem használható.

• A helyes válasz betűjelét karikázza be, vagy húzza alá. Javítani nem lehet.

I. Git alapok és mindennapi munka — 50 pont

25 feladat, feladatonként 2 pont

1. Mit hoz létre a git init parancs az aktuális mappában? (2 pont)

a) Egy üres, helyi repository-t a rejtett .git mappával
b) Egy üres commitot a munka megkezdéséhez
c) Egy kapcsolatot a megadott címen elérhető távoli tárolóval
d) Egy új ágat a meglévő repository-ban

2. Mit sorol fel a git status kimenetében az „Untracked files” szakasz? (2 pont)

a) A .gitignore állományban felsorolt kihagyott fájlokat
b) A legutóbbi commit óta törölt fájlok listáját
c) A mappában lévő, verziókövetésbe még fel nem vett fájlokat
d) A módosított, de még nem commitolt követett fájlok listáját

3. Mit eredményez az alábbi parancs? (2 pont)

```
git add .
```

a) Azonnal rögzíti az összes módosítást egy új commitban üzenet nélkül
b) Az aktuális mappa összes változását az előkészítő területre teszi
c) Hozzáadja a mappát a távoli tárolóhoz követésre
d) Visszaállítja a mappa fájljait az utolsó commit állapotára

4. Mi az index (staging area) szerepe a Gitben? (2 pont)

a) A fájlok korábbi változatait őrző gyorsítótár
b) A commitok sorrendjét nyilvántartó segédállomány
c) A távoli tárolóból letöltött commitok átmeneti tára
d) A következő commitba kerülő változások gyűjtőhelye

5. Mit végez az alábbi parancs? (2 pont)

```
git commit -m "Hibajavítás a számlázásban"
```

a) Rögzíti a munkakönyvtár összes változását előkészítés nélkül
b) Módosítja az utolsó commit üzenetét a megadottra
c) Feltölti a változásokat a távoli tárolóba üzenettel
d) Rögzíti az előkészített változásokat a megadott üzenettel

6. Miben más a git commit -a -m "..." a sima commitnál? (2 pont)

a) Minden ágon egyszerre készíti el ugyanazt a commitot
b) A követett fájlok módosításait külön add nélkül is rögzíti
c) Az új, még nem követett fájlokat is bevonja a készülő commitba
d) A commitot automatikusan fel is tölti a távoli tárolóba

7. Mit jelent, ha egy fájl állapota „modified”? (2 pont)

a) Olyan fájl, amely már bekerült az előkészítő területre
b) Olyan fájl, amelyet a .gitignore kizár a követésből
c) Követett fájl, amely a legutóbbi commit óta megváltozott
d) Olyan fájl, amelyet másik ágon is szerkesztettek

8. Mit eredményez az alábbi parancs? (2 pont)

```
git restore szamla.php
```

a) A fájl visszaáll a legutóbbi commit szerinti tartalomra
b) A fájl korábbi változatai megjelennek a naplóban
c) A fájl kikerül a verziókövetésből, de a lemezen marad
d) A fájl bekerül az előkészítő területre commitoláshoz

9. Mit eredményez az alábbi parancs? (2 pont)

```
git rm --cached beallitas.env
```

a) Kiveszi a fájlt a verziókövetésből, de a lemezen meghagyja
b) Kizárja a fájlt a következő commitból, de követi tovább
c) Visszaállítja a fájlt a legutóbbi commit állapotára
d) Törli a fájlt a lemezről és a verziókövetésből is, véglegesen

10. Mit végez a git mv parancs? (2 pont)

a) Átnevezi az ágat a megadott új névre a tárolóban
b) Átmozgatja a fájlt a munkakönyvtárból a gyorsítótárba
c) Átnevezi vagy áthelyezi a fájlt, és rögzíti a változást az indexben
d) Áthelyezi a megadott commitot egy másik ágra a repository előzményében

11. Mit zár ki a verziókövetésből az alábbi .gitignore sor? (2 pont)

```
*.log
```

a) A gyökérmappa egyetlen, log nevű állományát
b) A minden mappában lévő, .log kiterjesztésű fájlokat
c) A .log kiterjesztésű fájlokat, de csak a gyökérben
d) A log nevű mappát és annak teljes tartalmát

12. Miért nem hat a .gitignore egy már követett fájlra? (2 pont)

a) A szabály csak a következő ág létrehozásakor lép életbe
b) A szabály csak a bináris fájlokra alkalmazható
c) A szabály kizárólag a távoli tárolóban érvényesül
d) A szabály csak a még nem követett fájlokra vonatkozik

13. Mit jelenít meg az alábbi parancs? (2 pont)

```
git log --oneline --graph --all
```

a) A commitokat szerzők szerint csoportosítva
b) Az aktuális ág commitjait a teljes változáslistával együtt
c) Az összes ág commitjait rövid formában, ágrajzzal együtt
d) A távoli tárolóban lévő ágak összevont naplóját

14. Mit mutat meg a git show parancs egy commit azonosítójával? (2 pont)

a) A commit adatait és a benne szereplő változásokat
b) A commitot tartalmazó ágak teljes listáját
c) A commit óta eltelt összes módosítást a fájlokban
d) A commithoz tartozó pull request leírását

15. Mit jelent a commitok rövidített azonosítója, például 3f2a1b9? (2 pont)

a) A commitot készítő fejlesztő azonosítója
b) A commit időbélyegéből képzett rövid kód
c) A commit sorszáma az aktuális ágon belül, egytől számozva
d) A teljes SHA-azonosító eleje, amely egyedileg azonosít

16. Mit hasonlít össze a git diff parancs paraméter nélkül? (2 pont)

a) A két legutóbbi commit tartalmát egymással
b) A munkakönyvtárat az előkészítő terület tartalmával
c) Az előkészítő területet a legutóbbi commit tartalmával
d) A helyi ágat a megfelelő távoli ág állapotával

17. Mit mutat a git diff --staged parancs? (2 pont)

a) Az előkészített változásokat a legutóbbi commithoz képest
b) A még elő nem készített módosításokat mutatja a fájlokban
c) Az összes változást a repository létrejötte óta
d) A távoli ág és a helyi ág közötti különbséget

18. Mire való a git commit --amend? (2 pont)

a) Több commit egyetlen committá vonására
b) A commit áthelyezésére egy másik ágra
c) Az utolsó commit üzenetének vagy tartalmának módosítására
d) Egy korábbi commit visszavonására egy új, ellentétes committal

19. Miért kockázatos az --amend egy már feltöltött commitnál? (2 pont)

a) A módosított fájlok tartalma visszaállíthatatlan lesz
b) Átírja az előzményt, ezért a többieknél eltérés keletkezik
c) A távoli tároló automatikusan törli az ágat
d) A commit üzenete véglegesen elveszik a tárolóból és a naplóból

20. Mit eredményez az alábbi parancs? (2 pont)

```
git revert 9a3c17d
```

a) Visszaállítja a munkakönyvtárat a megadott commitra
b) Áthelyezi a megadott commitot az aktuális ág végére
c) Törli a megadott commitot az ág előzményéből, nyom nélkül eltüntetve
d) Új commitot készít, amely visszavonja a megadott commit változásait

21. Mi a lényegi különbség a git revert és a git reset között? (2 pont)

a) A revert kizárólag távoli, a reset helyi ágon működik
b) A revert megőrzi az előzményt, a reset átírja azt
c) A revert csak egy fájlra, a reset az egész ágra hat
d) A revert az indexet, a reset a munkakönyvtárat érinti

22. Mi történik az alábbi parancs hatására? (2 pont)

```
git reset --soft HEAD~1
```

a) Az utolsó commit és a hozzá tartozó változások is elvesznek
b) Az utolsó commit megmarad, csak az üzenete törlődik
c) Az ág visszaáll a távoli tároló állapotára
d) Az utolsó commit megszűnik, a változások az indexben maradnak

23. Mit tart nyilván a git reflog? (2 pont)

a) A HEAD korábbi állásait, így elveszettnek hitt commitok is megtalálhatók
b) A fájlonkénti módosítások időrendi listáját
c) A távoli tárolóból letöltött commitok naplóját áganként, időrendbe bontva
d) A repository beállításainak változástörténetét

24. Mit tartalmaz a projekt mappájában lévő .git könyvtár? (2 pont)

a) A távoli tárolóval folytatott kommunikáció naplóját
b) A verziókövetésből kihagyott fájlok biztonsági másolatát
c) A teljes verziótörténetet, az objektumokat és a beállításokat
d) Kizárólag a legutóbbi commit fájljainak teljes, tömörítetlen másolatát

25. Mire szolgál az alábbi parancs? (2 pont)

```
git config --global user.email "a@b.hu"
```

a) A commitok digitális aláírásának bekapcsolására
b) Az értesítő levelek címzettjének megadására
c) A távoli tárolóba való belépés hitelesítésére szolgál
d) A commitokhoz rögzített szerzői e-mail cím beállítására

II. Ágak, együttműködés és haladó eszközök — 50 pont

25 feladat, feladatonként 2 pont

1. Mit eredményez az alábbi parancs? (2 pont)

```
git switch -c szamlazas
```

a) Átvált a már létező szamlazas ágra
b) Létrehozza az ágat, de a jelenlegin marad
c) Átnevezi az aktuális ágat szamlazas névre
d) Létrehozza a szamlazas ágat, és átvált rá

2. Mit listáz a git branch -a parancs? (2 pont)

a) Az összes ág utolsó commitjának üzenetét
b) Kizárólag a helyi ágakat a tárolóban
c) A helyi és a távolról ismert ágakat egyaránt
d) A már egyesített, törölhető ágak listáját

3. Mit jelent, hogy egy helyi ág követ (tracking) egy távoli ágat? (2 pont)

a) A push és a pull alapértelmezett célja az a távoli ág lesz
b) A helyi ágon nem lehet commitolni, csak olvasni
c) A távoli ág minden commitja megjelenik a naplóban
d) A helyi ág automatikusan frissül a távoli ág változásaival

4. Mit végez az alábbi parancs? (2 pont)

```
git push -u origin szamlazas
```

a) Feltölti az összes helyi ágat a távoli tárolóba
b) Feltölti az ágat, és beállítja hozzá a követést
c) Létrehozza az ágat a távoli tárolóban, de nem tölt fel semmit
d) Letölti a távoli ág változásait a helyi ágba

5. Mit jelöl az origin név egy klónozott repository-ban? (2 pont)

a) A forrásként használt távoli tároló alapértelmezett neve
b) A repository legelső commitjának rövidített azonosítója
c) A repository tulajdonosának felhasználóneve
d) A főág neve a távoli tárolóban

6. Miben más a git pull --rebase a sima pullnál? (2 pont)

a) Eldobja a helyi commitokat a távoli javára
b) Egyesítő commitot készít a két ág összefésülésekor
c) A helyi commitokat a letöltött változások tetejére játssza újra
d) Csak letölti a változásokat, de nem egyesíti őket a helyi ággal

7. Miért utasítja el a szerver a push-t „non-fast-forward” hibával? (2 pont)

a) A helyi ág neve eltér a távoli tárolóban lévő ág nevétől
b) A távoli ágon olyan commit van, amely helyben még hiányzik
c) A commit üzenete nem felel meg a szabályoknak
d) A felhasználónak nincs joga az ágra írni

8. Hogyan zárunk le helyesen egy ütközés feloldását? (2 pont)

a) Az egyesítést megszakítjuk, és újra push-olunk
b) A javított fájlokat elmentjük, a commit automatikus
c) Az ütközést tartalmazó fájlt töröljük a tárolóból
d) A javított fájlokat hozzáadjuk, majd commitoljuk

9. Mit jelölnek az alábbi sorok egy ütköző fájlban? (2 pont)

```
<<<<<<< HEAD
   a mi változatunk
=======
   a másik ág változata
>>>>>>> szamlazas
```

a) Az ütköző szakasz két eltérő változatát
b) A fájl két, egymás után következő szakaszát
c) A commit előtti és utáni állapotot a fájlban
d) A fájl régi és új nevét az átnevezés után

10. Mit biztosít a git merge --no-ff kapcsoló? (2 pont)

a) Az egyesítés csak akkor fut le, ha nincs benne ütközés
b) Az egyesítés után az ág automatikusan törlődik
c) Mindig készül egyesítő commit, így látszik az ág nyoma
d) Az ág commitjai egyetlen committá vonódnak össze

11. Mit eredményez a git merge --abort parancs? (2 pont)

a) Felfüggeszti az egyesítést, a jelölések maradnak
b) Befejezi az egyesítést, a mi águnk javára döntve az ütközést
c) Törli az egyesítés során keletkezett új commitot
d) Megszakítja az egyesítést, és visszaáll az előtte lévő állapot

12. Mi a különbség a git branch -d és a -D kapcsoló között? (2 pont)

a) A kis d az ág utolsó commitját, a nagy D az egészet törli
b) A kis d csak a már egyesített ágat törli, a nagy D mindenképp
c) A kis d csak megjelöli, a nagy D ténylegesen törli az ágat
d) A kis d a helyi, a nagy D a távoli tárolóban lévő ágat törli

13. Mit jelent a squash egyesítés? (2 pont)

a) Az ág commitjait egyetlen committá vonja össze a célágon
b) Az ágat egyesítés nélkül, változtatás nélkül zárja le
c) Az ág minden commitját külön-külön átemeli a célágra
d) Az ág commitjait időrendbe rendezi az egyesítés előtt a célágon

14. Mire való az interaktív újrajátszás (git rebase -i)? (2 pont)

a) Két ág párhuzamos egyesítésére, commitonként jóváhagyva a lépéseket
b) A commitok feltöltésére egyesével a távoli tárolóba
c) Commitok átírására, összevonására és sorrendjük módosítására
d) Az ütközések automatikus feloldására szabályok alapján

15. Mi a rebase használatának alapszabálya? (2 pont)

a) Rebase után kötelező új címkét készíteni
b) Megosztott, már feltöltött ágon ne írjuk át az előzményt
c) Rebase előtt mindig törölni kell a helyi ágat a tárolóból
d) Rebase kizárólag a főágon végezhető el

16. Mit végez a git fetch --prune parancs? (2 pont)

a) Tömöríti a repository objektumait a helytakarékosság érdekében
b) Törli a helyi ágakat, amelyek már egyesültek a főággal, majd tömörít is
c) Eltávolítja a régi commitokat a megadott dátum előttről
d) Letölti a változásokat, és törli a megszűnt távoli ágak nyilvántartását

17. Mikor célszerű forkot készíteni egy GitHub projektről? (2 pont)

a) Ha a projektet teljesen le akarjuk másolni a gépünkre
b) Ha a projekt egyik ágán akarunk dolgozni a csapatban
c) Ha a projekt előzményét szeretnénk megőrizni archívumban
d) Ha nincs írási jogunk, de szeretnénk közreműködni benne

18. Mit jelent a piszkozat (draft) pull request? (2 pont)

a) Olyan kérés, amelyet a rendszer ütközés esetén automatikusan elutasít
b) Még nem kész változat, amely nem olvasztható be, de látható
c) Olyan kérés, amely ütközés miatt nem egyesíthető
d) Olyan kérés, amely csak a szerzője számára látható

19. Mire való a védett ág (protected branch) beállítás? (2 pont)

a) Megakadályozza a közvetlen push-t és feltételekhez köti a beolvasztást
b) Megakadályozza az ág klónozását külső felhasználóknak
c) Automatikus mentést készít az ág minden commitjáról
d) Titkosítja az ágon tárolt forráskódot a távoli tárolóban és a klónokban

20. Mit eredményez az alábbi parancs? (2 pont)

```
git tag -a v1.2.0 -m "Tavaszi kiadás"
```

a) Feltölti a megjelölt commitot a távoli tárolóba
b) Átnevezi az aktuális commitot a megadott verziószámra
c) Megjegyzéssel ellátott címkét tesz az aktuális commitra
d) Új ágat hoz létre v1.2.0 néven az aktuális commit fölött

21. Miért szükséges külön a git push --tags parancs? (2 pont)

a) Mert a címkéket a szerver nem fogadja el commit nélkül
b) Mert a címkéket csak a tároló gazdája töltheti fel
c) Mert a sima push nem viszi fel a helyi címkéket
d) Mert a címkék külön ágon tárolódnak a tárolóban

22. Mire szolgál a submodule egy repository-ban? (2 pont)

a) A repository egy mappájának külön ágon tartására
b) Másik repository beágyazására, adott commitra rögzítve
c) A nagy bináris fájlok külön tárolására hivatkozással
d) A függőségek verzióinak nyilvántartására szöveges fájlban

23. Mi a sekély (shallow) klónozás hátránya? (2 pont)

a) Hiányzik az előzmény, így egyes műveletek nem futtathatók
b) A klón nem köthető össze a távoli tárolóval
c) A letöltött fájlok tartalma tömörítve marad a munkakönyvtárban
d) A klónban nem lehet új ágat létrehozni

24. Mire való a .gitattributes állomány? (2 pont)

a) A fejlesztők jogosultságainak beállítására ágakon
b) A commit üzenetek kötelező formátumának leírására
c) A verziókövetésből kihagyandó fájlok és mappák felsorolására
d) Fájlonkénti kezelési szabályok, például a sorvégek megadására

25. Mi a Git LFS (Large File Storage) célja? (2 pont)

a) A bináris fájlok változásait szövegesen hasonlítja össze
b) A nagy fájlokat automatikusan kisebb darabokra bontja
c) A nagy bináris fájlokat külön tárolja, a repository-ban csak hivatkozás marad
d) A repository teljes előzményét tömörítve archiválja a szerveren és a klónban